

Trabalho Prático 3 – Testes Unitários e Exceções

Continuação do Sistema de Biblioteca em Python

Data 19 de junho de 2026
Disciplina Programação em Python
Pontuação 5,0 pontos

1 Objetivo

O objetivo deste trabalho é exercitar dois conceitos essenciais para a escrita de código confiável: **tratamento de exceções** e **testes unitários**. Você deverá dar continuidade ao sistema de biblioteca desenvolvido no TP2, aplicando agora novos conceitos a fim de torná-lo mais robusto por meio do tratamento adequado de exceções e de comprovar seu funcionamento por meio de testes automatizados.

Ao final da atividade, espera-se que você seja capaz de:

- sinalizar erros por meio de exceções (`raise`), em vez de apenas imprimir mensagens;
- criar exceções próprias para representar regras do domínio;
- tratar erros com blocos `try/except`, mantendo o programa em funcionamento;
- escrever testes automatizados com o módulo `unittest`;
- verificar tanto o comportamento correto quanto o comportamento de erro (`assertRaises`).

2 Ponto de partida

Para este trabalho, você utilizará o **sistema de biblioteca desenvolvido no TP2**. Não é necessário reescrever toda a solução: basta que as classes principais (`Livro`, `Usuario` e `Biblioteca`) e as funcionalidades de empréstimo e devolução estejam implementadas, pois será sobre elas que você aplicará o tratamento de exceções e escreverá os testes.

Caso sua solução do TP2 esteja incompleta, faça os ajustes necessários para que as operações de cadastro, empréstimo e devolução funcionem corretamente, pois elas servirão de base para esta entrega.

2.1 Por que usar exceções?

No TP2, quando uma operação era recusada (por exemplo, ao tentar emprestar um livro indisponível), o sistema apenas **exibia uma mensagem** na tela. O problema é que quem chamou o método não consegue saber, *em código*, que a operação falhou e o programa continua sua execução como se tudo tivesse ocorrido normalmente.

As exceções resolvem esse problema: elas **interrompem o fluxo de execução**, descrevem a causa do erro e podem ser **tratadas** (ou **testadas**) por quem chamou o método. É essa mudança de comportamento que você implementará na Parte 1.

3 Parte 1 — Tratamento de exceções

Refatore o seu sistema para sinalizar erros por meio de exceções.

- Crie **pelo menos 3 exceções próprias** (classes que herdam de `Exception`), com nomes que descrevam o erro. Exemplos: `LivroIndisponivelError`, `UsuarioInativoError`, `LivroNaoEncontradoError`, `CadastroDuplicadoError`.
- Escolha **pelo menos 3 métodos** do seu sistema e substitua as mensagens de erro por `raise` da exceção adequada.
- No programa principal, use `try/except` para **capturar** essas exceções e exibir uma mensagem amigável, demonstrando que o programa **continua funcionando** após um erro.

- **Pelo menos 3** dos lançamentos devem usar as suas exceções próprias. Em lançamentos adicionais, você também pode usar exceções embutidas (como `ValueError`), quando fizer sentido.

Alguns métodos candidatos:

Método	Quando lançar uma exceção
<code>emprestar_livro</code>	livro indisponível, usuário inativo, ou livro/usuário inexistente
<code>devolver_livro</code>	livro não estava emprestado, ou não foi encontrado
<code>adicionar_livro</code>	já existe um livro com o mesmo código
<code>cadastrar_usuario</code>	já existe um usuário com a mesma matrícula

Você pode escolher quaisquer métodos para proteger. O importante é que cada exceção represente uma regra real do sistema.

4 Parte 2 — Testes unitários

Crie um arquivo de testes (por exemplo, `test_biblioteca.py`) usando o módulo `unittest`.

- Crie uma classe de teste que herda de `unittest.TestCase`.
- Use `setUp()` para montar um cenário (uma biblioteca com alguns livros e usuários) **antes de cada teste** — assim você evita repetir código.
- Escreva **pelo menos 6 métodos de teste** (com nomes começando por `test_`), sendo:
 - **pelo menos 3 de casos válidos** (caminho feliz), usando asserções como `assertEqual`, `assertTrue`, `assertFalse` ou `assertIn`;
 - **pelo menos 2 de casos de erro**, usando `assertRaises` para verificar que as exceções da Parte 1 são realmente lançadas.
- Use **pelo menos 3 tipos diferentes** de asserção ao longo da suíte.
- A suíte deve ser executável pela linha de comando.

Assim como nas exceções, você decide quais métodos vai testar. Teste o que tem regras interessantes, operações que mudam o estado do sistema e operações que devem falhar.

5 Exemplos

Os trechos abaixo ilustram o que se espera. Os nomes podem ser adaptados ao seu código, desde que o comportamento seja equivalente.

5.1 Definindo e lançando uma exceção

```
class LivroIndisponivelError(Exception):
    pass

class Biblioteca:
    def emprestar_livro(self, codigo_livro, matricula):
        livro = self.buscar_por_codigo(codigo_livro)
        if not livro.disponivel:
            raise LivroIndisponivelError(
                f"O livro '{livro.titulo}' nao esta disponivel."
            )
        # ... segue com o emprestimo
```

5.2 Tratando a exceção no programa principal

```
try:
    biblioteca.emprestar_livro("L001", "U002")
except LivroIndisponivelError as erro:
    print(f"Emprestimo recusado: {erro}")
```

5.3 Testando com unittest (um caso válido e um caso de erro)

```
import unittest
from biblioteca import Biblioteca, Livro, Usuario, LivroIndisponivelError

class TestBiblioteca(unittest.TestCase):
    def setUp(self):
        self.biblioteca = Biblioteca("Biblioteca CEFET")
        self.biblioteca.adicionar_livro(Livro("L001", "1984", "Orwell", 1949))
        self.biblioteca.cadastrar_usuario(Usuario("U001", "Ana", "ana@email.com"))

    def test_emprestimo_deixa_livro_indisponivel(self):
        self.biblioteca.emprestar_livro("L001", "U001")
        livros = self.biblioteca.buscar_livros_por_titulo("1984")
        self.assertFalse(livros[0].disponivel)

    def test_emprestar_indisponivel_lanca_excecao(self):
        self.biblioteca.emprestar_livro("L001", "U001")
        with self.assertRaises(LivroIndisponivelError):
            self.biblioteca.emprestar_livro("L001", "U001")

if __name__ == "__main__":
    unittest.main()
```

A linha de `import` depende de como você organizou os arquivos: importe as classes (e as exceções) a partir do(s) módulo(s) do seu sistema.

6 Executando os testes

Rode a suíte pelo terminal, a partir da pasta do projeto:

```
python -m unittest test_biblioteca.py -v
```

Leia o resultado: cada `.` é um teste aprovado; `F` indica uma falha de asserção (o valor obtido foi diferente do esperado); `E` indica um erro inesperado durante o teste. **A sua entrega deve terminar com OK.**

7 Organização da entrega

- Sua entrega do TP2, agora com as exceções (incluindo o programa de demonstração, agora com `try/except`).
- O arquivo de testes (por exemplo, `test_biblioteca.py`).

O arquivo de testes deve **importar** as classes do seu sistema — não copie o código das classes para dentro dele.

8 Fora do escopo

- **Mocks** (`unittest.mock` / `@patch`) não são obrigatórios. Ficam como desafio opcional para quem quiser explorar o que vimos em aula.
- Não é necessário usar `pytest`; use o `unittest`.

- Não é preciso testar todo o sistema: foque nos métodos com regras relevantes.
- Continuem fora do escopo: interface gráfica, banco de dados e leitura/escrita de arquivos.

9 Critérios de avaliação

A nota total será de 5 pontos.

Critério	Pts	Descrição
Exceções próprias	1,0	Criou pelo menos três classes de exceção próprias, com nomes claros e coerentes.
Lançamento de exceções	1,0	Substituiu mensagens por <code>raise</code> em pelo menos três métodos, conforme as regras do sistema.
Tratamento com <code>try/except</code>	0,5	O programa captura as exceções e continua funcionando após um erro.
Estrutura dos testes	1,0	Usa <code>unittest.TestCase</code> , <code>setUp()</code> e nomes <code>test_*</code> ; a suíte roda pela linha de comando.
Cobertura dos testes	1,0	Pelo menos seis testes: casos válidos (com asserções variadas) e casos de erro com <code>assertRaises</code> .
Qualidade e clareza	0,5	Nomes descritivos, testes legíveis e código organizado.
Total	5,0	

10 Formato de entrega

- Enviar os arquivos `.py` (o sistema e o arquivo de testes).
- Os testes devem rodar com `python -m unittest`, sem dependências externas.
- O estudante deve entender o código entregue e ser capaz de explicá-lo.

10.1 Observação sobre originalidade

Plágio zera a atividade. O uso de IA generativa é permitido como apoio ao estudo, desde que o estudante revise, compreenda e consiga explicar a solução entregue.

Em caso de dúvidas, procure o professor antes do prazo de entrega.